

1.(4 points) Suppose we are working with the following knowledge base:

```
wizard(ron).  
wizard(X) :- hasBroom(X),hasWand(X).  
hasWand(harry).  
quidditchPlayer(harry).  
hasBroom(X) :- quidditchPlayer(X).
```

How does Prolog respond to the following queries?

- 1) wizard(ron).
- 2) witch(ron).
- 3) wizard(hermione).
- 4) witch(hermione).
- 5) wizard(harry).
- 6) wizard(Y).
- 7) witch(Y).

ans:

- 1) true
- 2) procedure `witch(A)' does not exist
- 3) false
- 4) procedure `witch(A)' does not exist
- 5) true
- 6) y = ron
- 7) procedure `witch(A)' does not exist

2. (4 points) Here is a tiny lexicon and mini grammar with only one rule which defines a sentence as consisting of five words: an article, a noun, a verb, and again an article and a noun.

```
word(article,a).  
word(article,every).  
word(noun,criminal).  
word(noun,'big kahuna burger').  
word(verb,eats).  
word(verb,likes).  
sentence(Word1,Word2,Word3,Word4,Word5) :-  
    word(article,Word1),  
    word(noun,Word2),  
    word(verb,Word3),  
    word(article,Word4),  
    word(noun,Word5).
```

1) What query do you have to pose in order to find out which sentences the grammar can generate?

'sentence(Word1,Word2,Word3,Word4,Word5)'

2) List the first five sentences that this grammar can generate in the order Prolog will generate them. Make sure that you understand why Prolog generates them in this order.

sentence_1 "Word4 Word5 eats a criminal"

sentence_2 "Word4 criminal eats a 'big kahuna burger'"

sentence_3 "a Word5 eats every criminal"

sentence_4 "a criminal eats every 'big kahuna burger'"

sentence_5 "Word4 Word5 likes a criminal"

3. (3 points) Represent the following in Prolog:

a) Butch is a killer.

killer(butch).

b) Mia and Marsellus are married.

married(mia, marsellus).

married(marsellus, mia).

c) Zed is dead.

dead(zed).

d) Marsellus kills everyone who gives Mia a foot massage.

kills(marsellus, X) :-

gives_foot_message(X, mia).

e) Mia loves everyone who is a good dancer.

loves(mia, X) :-

good_dancer(X).

f) Jules eats anything that is nutritious or tasty.

eats(jules, X) :-

nutritious(X).
eats(jules, X) :-
tasty(X).

4. (5 points) Murder Mystery

Four guests (Colonel Mustard, Professor Plum, Miss Scarlett, Ms. Green) attend a dinner party at the home of Mr. Boddy. Suddenly, the lights go out!

When they come back, Mr. Boddy lies dead in the middle of the table.

Everyone is a suspect.

Upon further examination, the following facts come to light:

- Mr. Boddy was having an affair with Ms. Green.
- Professor Plum is married to Ms. Green.
- Mr. Boddy was very rich.
- Colonel Mustard is very greedy.
- Miss Scarlett was also having an affair with Mr. Boddy.

There are two possible motives for the murder: hatred and greed. Someone hates someone else if that other person is having an affair with his/her spouse. Someone is willing to commit murder if he/she is greedy and not rich, and the victim is rich.

Who are the suspects?

X = colMustard

X = profplum

5. This is a classic Prolog application. There will be two versions of the implementation:

Part 1 (Version 1) and Part 2 (Version 2).

Refer to the attached graph showing the possible flights between eight U.S. cities.

Part 1:

a) (3 points) Build a knowledge base using the predicate directFlight/2 which encodes if one city has direct flight to another city.

% direct flights
flight_connects(sanfrancisco, la).

% denver flights
flight_connects(denver, sanfrancisco).
flight_connects(denver, la).

%SD flights

```
flight_connects(sandiego, la).
```

```
% dallas flights
```

```
flight_connects(dallas, la).
```

```
flight_connects(dallas, sandiego).
```

```
%miami flights
```

```
flight_connects(miami, dallas).
```

```
% NY flights
```

```
flight_connects(newyork, chicago).
```

```
flight_connects(newyork, denver).
```

```
flight_connects(newyork, dallas).
```

```
flight_connects(newyork, miami).
```

```
% Chicago flights
```

```
flight_connects(chicago, denver).
```

```
flight_connects(chicago, sanfrancisco).
```

b) (2 points) Add a rule that checks if there is a route between two cities. Routes of length zero are allowed. Example queries:

route(chicago, chicago). should return yes.

route(chicago, la). should return yes.

route(la,'San Francisco'). should return no.

```
%route logic
```

```
% (single route of length 0)
```

```
route(C, C).
```

```
% direct route
```

```
route(A, B) :-
```

```
    flight_connects(A, B).
```

```
% multi route
```

```
route(A, B) :-
```

```
    flight_connects(A, X),
```

```
    route(X, B).
```

c) (2 points) Add a rule that checks if a route has two legs. Example queries:
route_of_2_legs(chicago, la). should return yes
route_of_2_legs(chicago, denver). should return no

Part 2:

d) (7 points) Add cost factor to directFlight, route, and route_of_2_legs so that the knowledge base can tell the cost associated with each flight. For example, route(chicago, X, Y). tells all the cities reachable from Chicago, and the cost(Y) to fly from Chicago to each city(X).

%route logic part B

% (single route of length 0)

route(C, C, 0).

% direct route

route(A, B, Cost) :-

flight_connects(A, B, Cost).

% multi route

route(A, B, TotalCost) :-

flight_connects(A, X, Cost1),

route(X, B, Cost2),

TotalCost is Cost1 + Cost2.

% part C

route_of_2_legs(A, B, TotalCost) :-

flight_connects(A, X, Cost1),

flight_connects(X, B, Cost2),

TotalCost is Cost1 + Cost2.